Bistabili, latch, flip-flop e sintesi di circuiti sequenziali

Marco Cerbella

Lezione 8

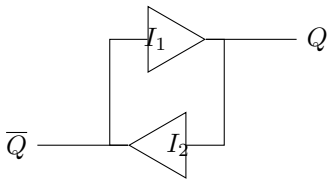
1 Elementi bistabili

Il blocco costitutivo fondamentale di un registro di memoria è un elemento *bistabile*: un elemento circuitale con due stati stabili, in grado dunque di memorizzare un bit di informazione. Un elemento con N stati stabili conserva $\log_2 N$ bit.

Definizione. [Stato stabile] Dato un circuito (sequenziale), si dice *stato stabile* del circuito un'attribuzione di valori di verità a tutti i fili del circuito in accordo con il comportamento delle porte logiche.

1.1 Due negatori collegati a croce

Il bistabile più semplice è formato da una coppia di negatori I_1, I_2 collegati a croce: l'ingresso di I_1 è l'uscita di I_2 e viceversa.



L'analisi di un elemento sequenziale differisce da quella di un circuito combinatorio perché il primo è *ciclico*: le uscite Q e $\overline{Q}$ dipendono l'una dall'altra. Si studiano i due casi $Q = 0$ e $Q = 1$.

- **Caso $Q = 0$.** I_2 riceve in ingresso 0 e produce 1 su $\overline{Q}$; I_1 riceve 1 da $\overline{Q}$ e produce 0 su Q. Lo stato è stabile perché viene confermata la premessa iniziale ($Q = 0$).
- **Caso $Q = 1$.** I_2 riceve 1 e produce 0 su $\overline{Q}$; I_1 riceve 0 e produce 1 su Q. Anche questo stato è stabile.

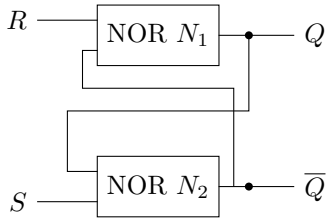
Osservazione. Il circuito ha in realtà anche un terzo possibile stato, con entrambe le uscite circa a metà tra 0 e 1: è detto stato *metastabile*. Inoltre, la variabile di stato è una sola: $\overline{Q}$ non contiene informazione aggiuntiva, essendo il complemento di Q.

Limiti. Due negatori collegati a croce memorizzano un bit, e un insieme di coppie di negatori potrebbe codificare le variabili di stato di una macchina a stati finiti; tuttavia l'utente non ha a disposizione *alcun ingresso* per controllare tali variabili. I latch e i flip-flop che seguono sono provvisti di ingressi per controllare il valore delle variabili di stato, e sono quindi più utili per codificare gli stati di una FSM.

2 Latch

2.1 Latch SR

Il latch SR si realizza con due porte NOR N_1, N_2 collegate a croce. Ha due ingressi S (*Set*) e R (*Reset*) e due uscite Q e $\overline{Q}$.



Analisi.

- $S = 0$, $R = 0$. Nessun ingresso è attivo: l'uscita Q ricorda il suo valore precedente Q_{prec} (e $\overline{Q}$ il suo complemento). Vi sono due stati stabili, a seconda di Q_{prec}.
- $S = 1$, $R = 0$. N_2 riceve almeno un valore vero ($S = 1$) e produce 0 su $\overline{Q}$; a questo punto N_1 ha entrambi gli ingressi a 0 e produce $Q = 1$. Quando viene attivato S, Q viene *settato* a 1.
- $S = 0$, $R = 1$. N_1 riceve almeno un valore vero ($R = 1$) e produce 0 su Q; N_2 ha entrambi gli ingressi a 0 e produce $\overline{Q} = 1$. Quando viene attivato R, Q viene *resettato* a 0.
- $S = 1$, $R = 1$. N_1 e N_2 vedono entrambe almeno un ingresso a 1 e producono entrambe 0. Non ha senso: significa settare e resettare contemporaneamente, e Q e $\overline{Q}$ non sono l'una l'opposto dell'altra (inconsistenti).

S	R	Q	$\overline{Q}$
0	0	Q_{prec}	$\overline{Q}_{\text{prec}}$
0	1	0	1
1	0	1	0
1	1	0	0

Limiti. Il latch SR è scomodo per due motivi:

- si comporta in modo inconsistente quando S e R sono attivati contemporaneamente (entrambe le uscite valgono 0);
- gli ingressi S e R combinano gli aspetti del *come* e del *quando*: attivare uno di essi determina non solo come debba diventare lo stato Q, ma anche quando debba cambiare.

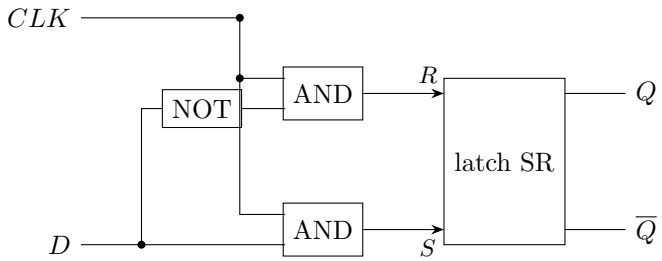
Questi limiti sono superati dal latch D.

2.2 Latch D

Il latch D ha due ingressi:

- un ingresso dati D, che controlla il *prossimo stato*;
- un ingresso di clock CLK, che controlla il *momento* del cambio di stato.

Si ottiene da un latch SR ponendo $S = D \cdot CLK$ e $R = \overline{D} \cdot CLK$.



Analisi.

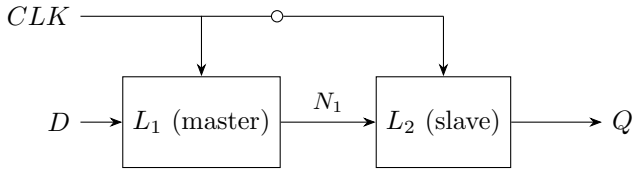
CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	$\times$	$\overline{\times}$	0	0	Q_{prec}	$\overline{Q}_{\text{prec}}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Quando $CLK = 0$ il latch ricorda lo stato precedente; quando $CLK = 1$ copia D su Q. Il caso $S = R = 1$ non può mai presentarsi, perché S e R sono sempre uno il complemento dell'altro quando $CLK = 1$.

Limiti. Il latch D aggiorna *continuamente* il suo stato finché $CLK = 1$, mentre sarebbe utile aggiornare lo stato solo in un momento preciso. Questo limite è superato dai flip-flop.

3 Flip-flop D

Un flip-flop D si realizza con due latch D in cascata, controllati da due segnali di clock complementari: il primo latch L_1 è detto *master*, il secondo L_2 *slave*; il nodo N_1 collega i due latch.



Analisi.

- Quando $CLK = 0$, il master è trasparente e qualsiasi valore di D viene portato in N_1; lo slave è opaco e mantiene l'uscita.
- Quando CLK diventa 1, il master si blocca e il valore di N_1 viene trasmesso sull'uscita dell'elemento (Q in L_2).

In altre parole, un flip-flop D *copia D su Q al fronte di salita del clock* e ricorda il suo stato in tutti gli altri casi.

Il simbolo circuitale del flip-flop D è un rettangolo con ingressi D e CLK (quest'ultimo contrassegnato da un triangolo, che indica la sensibilità al fronte) e uscite Q e $\overline{Q}$; esiste una versione semplificata con la sola uscita Q.

4 Registro di memoria

Definizione. [Registro a N bit] Un *registro* a N bit è un banco di N flip-flop che condividono un ingresso di clock CLK in comune, in modo che tutti i bit vengano aggiornati allo stesso tempo.

5 Sintesi di un circuito sequenziale: esempio

Esempio (Semaforo di un incrocio)**. Specifiche.**

- Incrocio tra due strade, nord-sud (NS) ed est-ovest (EO), controllate da un semaforo.
- Il semaforo può commutare ogni 30 secondi; per semplicità si considerano solo rosso e verde.
- Un sensore legge, per ogni direttrice, se esiste almeno un'auto in attesa oppure un'auto che si accinga ad attraversare (condizioni trattate allo stesso modo).
- Il semaforo deve cambiare colore, da rosso a verde, quando esiste un'auto in attesa sulla sua direttrice.
- Se ci sono auto in attesa su entrambe le direttrici il semaforo deve cambiare colore (al termine del tempo di commutazione).

Codifica. Il primo passo è modellare il circuito con una macchina a stati finiti.

- Gli ingressi sono codificati con due variabili booleane i_{EO}, i_{NS}.
- Serve 1 bit per codificare lo stato (due stati possibili): serve dunque un solo bistabile, un flip-flop D F con uscite Q_F, $\overline{Q}_F$. Si pone $Q_F = 1$ per lo stato VerdeNS e $Q_F = 0$ per VerdeEO.
- Serve 1 bit per l'uscita: $u = 1$ codifica LuceVerdeNS, $u = 0$ codifica LuceVerdeEO.

Tabella di transizione. D_F è lo stato futuro.

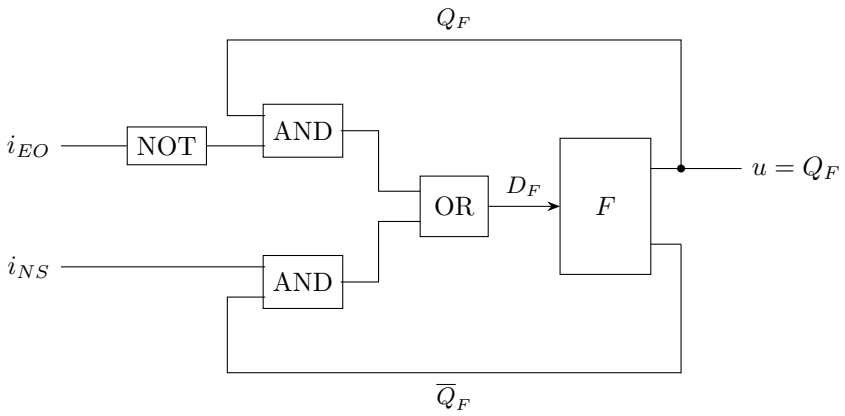
Q_F	i_{EO}	i_{NS}	D_F	u
1	0	0	1	1
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	1	0

Mappe di Karnaugh. Dalla tabella si ottengono

$$D_F = \overline{Q}_F\, i_{NS} + Q_F\, \overline{i_{EO}}, \qquad u = Q_F.$$

Infatti, per $Q_F = 0$ lo stato futuro vale 1 solo se $i_{NS} = 1$ (cioè $D_F = i_{NS}$), mentre per $Q_F = 1$ vale 1 solo se $i_{EO} = 0$ (cioè $D_F = \overline{i_{EO}}$). L'uscita coincide con lo stato.

Circuito. Il circuito si sintetizza con due porte AND, una porta OR e un negatore: il flip-flop D F ha in ingresso $D_F = \overline{Q}_F\, i_{NS} + Q_F\, \overline{i_{EO}}$ e l'uscita u è Q_F.



Osservazione. In sintesi, il procedimento per realizzare un circuito sequenziale è: modellare il problema con una FSM; codificare ingressi, stati e uscite con variabili booleane; scrivere la tabella di transizione (stato futuro e uscita in funzione di stato presente e ingressi); semplificare con le mappe di Karnaugh; realizzare la rete combinatoria e collegarla a un registro di flip-flop D che memorizza lo stato.

Riferimenti. Capitolo 3 del libro di supporto (S. Harris et al., *Sistemi Digitali e Architettura dei Calcolatori*), sezioni 3.1, 3.2, 3.2.1, 3.2.2, 3.2.3 e 3.2.4.